HAFTA

03

Tarayıcı ve HTML

Sunucudan gelen şey bir metin dosyasıydı. Bu hafta o metnin nasıl önce bir ağaca, sonra ekranda piksellere dönüştüğüne bakıyoruz.

BS435 · Web Programlama I

HAFTANIN SORUSU

Tarayıcı bir metin dosyasını nasıl ekrana çeviriyor?

Bu haftanın hedefleri

Her başlığın derinliği aynı değil

MUTLAKA BİL

HTML belge iskeletini, anlamsal öğeleri ve kaynak kod–DOM ayrımını açıkla.

DERSTE UYGULA

`F12 → Elements` panelinde DOM ağacını oku; başlık hiyerarşisini denetle.

MERAK EDİYORSAN İNCELE

Ayrıştırma sırası, tarayıcının hatalı HTML'i onarması ve CSSOM ilişkisi.

Kampüs panosu: elimizdeki tek şey bir metin dosyası

```
<h1>Kampüs Panosu</h1>
<article>
  <h2>Yeni laboratuvar açıldı</h2>
  <p>Tanıtım toplantısı 14.00'te.</p>
</article>
```

Sunucudan gelen tam olarak budur: biçimlendirilmemiş düz metin. Ekranda ise başlık büyük, duyuru onun altında, satırlar ayrı ayrı görünür. Aradaki dönüşümü bugün adım adım izleyeceğiz.

SORU

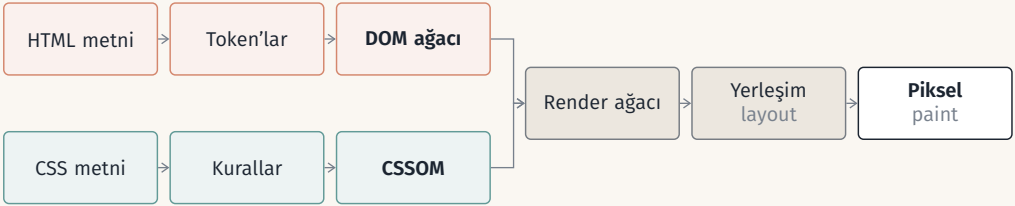
Başlığın ekranda büyük görünmesi, onun `h1` olduğu anlamına gelir mi?

1

Tarayıcının oluşturma süreci

Tarayıcı HTML ve CSS'i işleyerek sayfayı ekrana çizer.

Metinden piksele



Bu hafta soldaki üst şeridi işliyoruz: **HTML metninden DOM ağacına**. Alt şerit önümüzdeki haftanın konusu.

Sıra neden önemli?

- Tarayıcı HTML'i **yukarıdan aşağıya** okur ve okudukça DOM'u kurar.
- `defer` ya da `async` taşımayan bir `<script>` ile karşılaşırsa ayrıştırma **durur** — betik çalışmadan devam etmez.
- Çünkü betik DOM'u değiştirebilir; tarayıcı ne olacağını önceden bilemez.

Bu yüzden betikler geleneksel olarak sayfanın **sonuna** konur ya da `defer` ile işaretlenir: "beni sonra çalıştır, sen okumaya devam et".

→ **WP-II** React uygulamaları da sonunda bu hattan geçer; yalnızca DOM'u JavaScript kurar.

2

HTML’in yapısı

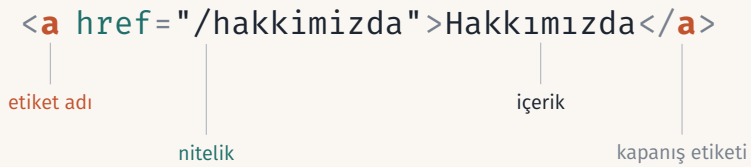
Her HTML belgesi aynı iskeleti paylaşır.

Belgenin iskeleti

```
<!doctype html>
<html lang="tr">
  <head>
    <meta charset="utf-8">
    <title>Kampüs Panosu</title>
  </head>
  <body>
    <h1>Kampüs Panosu</h1>
  </body>
</html>
```

- `head` ekranda görünmeyenler: başlık, kodlama, stil bağlantıları
- `body` kullanıcının gördüğü her şey
- `charset` karakter kodlaması — eksikse Türkçe harfler bozulur
- `lang` belgenin dili — ekran okuyucu ve çeviri bunu kullanır

Bir öğenin (element) anatomisi



Nitelikler (attribute) öğeye **ek bilgi** verir. Bazıları her öğede kullanılabilir (`id`, `class`, `lang`), bazıları yalnızca belirli öğelerde anlamlıdır (`href`, `src`, `alt`).

Sık kullanılan öğeler

METİN VE BAĞLANTI

- `h1`–`h6` başlık hiyerarşisi
- `p` paragraf (içinde `strong`, `em`)
- `a` bağlantı
- `ul`, `ol`, `li` listeler

İÇERİK VE YAPI

- `img` görsel (`alt` zorunlu sayın)
- `table` tablo (`tr`, `th`, `td`)
- `div` anlamsız kutu
- `span` anlamsız satır içi kutu

BAŞLIK HİYERARŞİSİ

`h1`–`h6` bir **içindekiler ağacıdır**, font boyutu ayarı değildir — atlamadan kullanın; görsel boyut CSS'in işidir.

3

Anlamlı HTML

Aynı görüntüyü veren iki belgeden biri makineler için anlamsızdır.

İki belge, aynı görüntü

ANLAMSIZ

```
<div class="ust">
  <div class="menu">...</div>
</div>
<div class="icerik">
  <div class="yazi">Haber</div>
</div>
```

ANLAMLI

```
<header>
  <nav>...</nav>
</header>
<main>
  <article>Haber</article>
</main>
```

Ekranda ikisi de aynı görünebilir. Ama sağdaki belge **ne olduğunu söylüyor**; soldaki yalnızca **nasıl göründüğünü** ima ediyor.

- Ekran okuyucu “ana içeriğe atla” derken `main` gibi bir işaret arar.
- Arama motoru `article` içindekini sayfanın asıl içeriği sayar.
- Altı ay sonra kodu okuyan siz de `<div class="k2">` ne demekti diye düşünmezsiniz.

Bir sayfanın anlamlı iskeleti



Bu öğeler **anlam taşır**; tarayıcı varsayılan stiller uygulayabilir. Görünüm önümüzdeki haftanın işi.

Birlikte çözelim: panoyu anlamlandıralım

ELİMİZDEKİ

```
<div class="ust">
  <div class="b">Kampüs Panosu</div>
</div>
<div class="icerik">
  <div class="duyuru">
    <div class="bas">Yeni lab</div>
    <div>Toplantı 14.00</div>
  </div>
</div>
```

HANGİ ÖĞE, NEDEN

`ust` → `header` sayfa başı
`b` → `h1` sayfanın başlığı
`icerik` → `main` asıl içerik
`duyuru` → `article` tek başına anlamlı
`bas` → `h2` duyurunun başlığı
son `div` → `p` paragraf

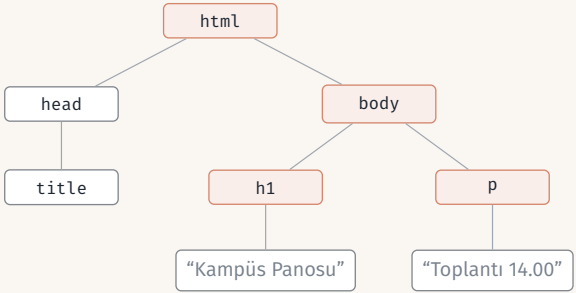
Öğeyi görünümüne göre değil, **sayfadaki rolüne göre** seçin — rolü olmayan bir kutu için `div` doğru tercihtir.

4

DOM

Metin bir belgedir; DOM programlanabilir bir nesnedir.

HTML bir metindir, DOM bir ağaçtır



Her öğe bir **düğüm**, her metin parçası da ayrı bir düğümdür — en alttaki iki kutu metin düğümüdür. JavaScript bu ağaçta gezinir, düğüm ekler, siler, değiştirir; tarayıcı ekranı buna göre günceller.

Kaynak kod ile DOM aynı şey değil

>_ CANLI DEMO

Aynı sayfayı iki yerde açalım: kaynak kodu (Ctrl+U) ve **F12 → Elements**. İkisi neden farklı görünüyor?

- Kaynak kod **sunucudan gelen metindir** — hiç değişmez.
- Elements sekmesi **canlı DOM'u** gösterir — JavaScript onu değiştirebilir.
- Eksik kapatılmış bir etiket kaynakta hatalı, DOM'da **düzeltilmiş** görünür: tarayıcı bozuk HTML'i onarmaya çalışır.

Sıra sizde

ALIŞTIRMA · 5 dk · tarayıcıda

Açık duran sayfada **F12 → Elements** panelini açın ve ağacı okuyun:

1. Sayfada kaç tane `h1` var? Başlıklar atlanmış mı?
2. `main` var mı, yerinde yalın bir `div` mi duruyor?
3. Bir `img` seçin: `alt` değeri ne diyor?

Aynı sayfayı anlamsal öğelerle yeniden yazma işi haftalık uygulamada: `examples/hafta-03-html`.

Haftanın sorusunun cevabı

1. Sunucudan **düz metin** gelir (`Content-Type: text/html`).
2. Tarayıcı metni soldan sağa, yukarıdan aşağıya **ayrıştırır**.
3. Etiketlerden **DOM ağacını** kurar; bozuk yapıları onarmaya çalışır.
4. CSS'ten ayrı bir ağaç (CSSOM) kurar ve ikisini birleştirir.
5. Yerleşimi hesaplar ve pikselleri **boyar**.

Bugün 1–3'ü işledik. **4 ve 5** önümüzdeki haftanın konusu.

Son bir soru: Elements panelinde yaptığınız bir değişiklik, sayfayı yenileyince neden kaybolur?

Nerede kaldık?

Üç hafta sonra: belge yapısı yerine oturdu

01 Ağ ve DNS	**02** HTTP	**03** HTML ve DOM	**04** CSS
05 Formlar	**06** Git ve yayın	**07** JavaScript	**08** DOM ve olaylar
09 Asenkron ve JSON	**10** Fetch ve API	**11** Durum ve önbellek	**12** Backend
13 Veritabanı	**14** Güvenlik		

Kiremit çerçeve: işlendi. Gri çerçeve: sonraki haftalar.

Önümüzdeki hafta

Ağaç kuruldu ama her şey üst üste, siyah beyaz duruyor. **Görünümü kim belirliyor?**

CSS'in nasıl karar verdiğini, kutu modelini ve aynı sayfanın telefonda neden farklı dizildiğini göreceğiz.

> **→ WEB PROGRAMLAMA II**
>
> React bileşenleri sonuçta bu DOM ağacını üretir. Bu haftayı bilmeden React'ın ne yaptığı anlaşılmaz.